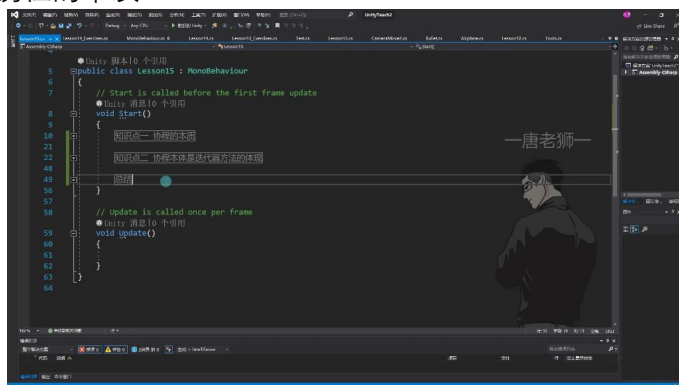


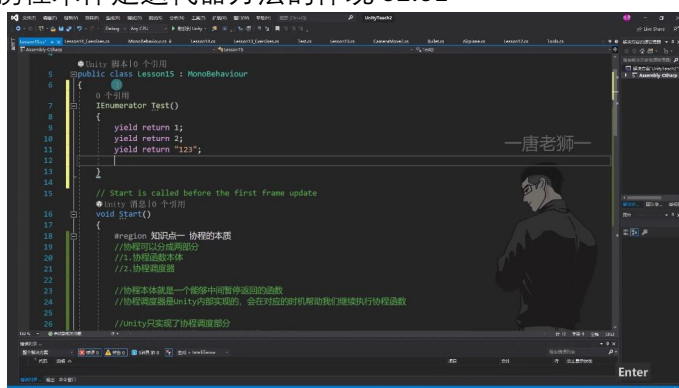
一、协同程序原理 00:00

1. 协同程序 00:21

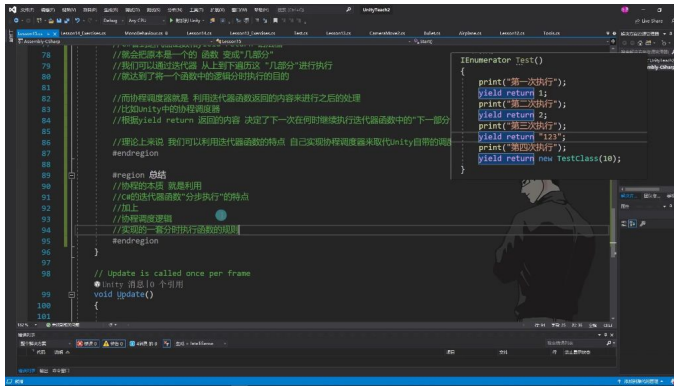
1) 携程的本质 00:33



- - **组成结构：**协同程序可以分成两部分
 - 1.协程函数本体
 - 2.协程调度器
 - **函数本体特性：**协程本体就是一个能够中间暂停返回的函数
 - **调度器功能：**协程调度器是Unity内部实现的，会在对应的时机帮助我们继续执行协程函数
 - **实现范围：**Unity只实现了协程调度部分，协程的本体本质上就是一个C#的迭代器方法
- #### 2) 携程本体是迭代器方法的体现 02:01



- - **手动执行原理：**
 - 如果不通过开启协程方法执行协程，Unity的协程调度器不会管理协程函数
 - 但可以自己执行迭代器函数内容
 - **迭代器工作机制：**
 - MoveNext()会执行函数中内容直到遇到yield return为止的逻辑
 - Current属性可以得到yield return返回的内容
 - MoveNext返回值代表是否到了结尾（迭代器函数是否执行完毕）
 - **执行示例：**
 - **调度器原理：**继承MonoBehavior后开启协程，相当于把协程函数（迭代器）放入Unity的协程调度器中管理执行
- #### 3) 总结 15:19



-
- **核心机制：**
 - 利用C#的迭代器函数"分步执行"的特点
 - 加上协程调度逻辑
 - 实现一套分时执行函数的规则
- **分步与分时：**
 - 迭代器函数实现"分布"概念
 - 协程调度器实现"分时"概念
- **自定义可能性：** 理论上可以利用迭代器函数特点自己实现协程调度器来取代Unity自带的调度器

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
协程的本质	协程由两部分组成： 1. 协程函数本体 （可暂停并返回的迭代器方法） 2. 协程调度器 （Unity内部实现，管理执行时机）	- 直接调用协程函数不会执行，需通过StartCoroutine触发 - 调度器逻辑由Unity实现 ，开发者仅需关注迭代器方法	★★★
协程与迭代器的关系	协程本质是C#迭代器方法： - 通过yield return分步执行 - IEnumerator.MoveNext()控制执行流程 - Current获取yield return返回值	- 手动执行迭代器 ：通过MoveNext()逐步触发逻辑 - 返回值类型 ：yield return可返回任意对象（如int、自定义类）	★★★★
Unity协程调度规则	Unity根据yield return内容决定后续执行时机： - 如yield return null等待下一帧 - 自定义类需Unity预定义规则处理	- 调度器不可见 ：Unity内部管理迭代器执行时机 - 关键方法 ：StartCoroutine将迭代器交给Unity管理	★★★
手动模拟协程调度	通过while循环+MoveNext()完整执行迭代器： - MoveNext()返回bool标识是否结束 - Current获取分步返回值	- 对比Unity调度器 ：手动调度缺少时间控制逻辑 - 易错点 ：忽略MoveNext()返回值导致无限循环	★★★★

协程的核心价值	实现函数逻辑的 分时执行 ： - 迭代器函数：分步执行（空间维度） - 调度器：控制时机（时间维度）	- 应用场景 ：延迟执行、异步任务、复杂状态机	★★★
----------------	---	--------------------------------	-----